

작업 계획

실험이 성립하기까지

인프라는 끝났다. 실험은 원본에서 끊긴 피드백 루프 하나 때문에 아직 시작되지 못했고, 그 위에 얹힌 개입 판정 공식은 위험을 재지 않아 논문의 핵심 지표가 지금은 무의미하다.

60~90%

SLA 위반 중
배분 탕

0.49

개입 판정의
SLA 예측력 (AUC)

3

코드 전에
정할 결정

지금 어디까지 왔나

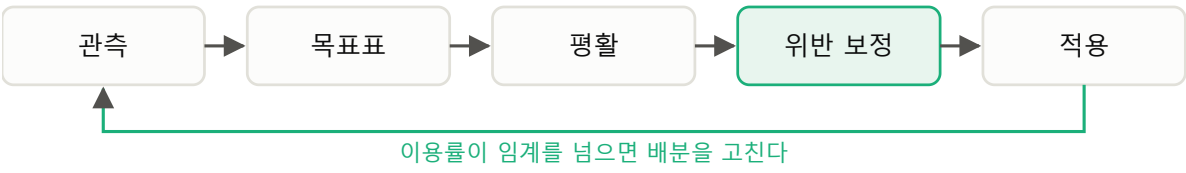
배선 · 서버 5개 · 드라이버 2종 · 채점기 · 재현성		완료
실험 장치 · 비교군 · 집계 · 배치		arms/ · run_matrix 없음
결정 · 위반 보정 · 개입 공식 · 조달 시점		회의 필요
결과 · 논문 표에 들어갈 수치		측정 불가 상태

"구축은 끝났고 나머지는 조정"은 절반만 맞다. 조정 중 하나(개입 판정 공식)가 논문의 축을 흔들 수 있다.

왜 지금 결과가 이런가

원본에서 끊긴 것

원본 ml_orchestrator_demo.py:429~461 (한 함수)



현재 ② rule.py + ① env.py (서버 두 개로 분리)



평활이 ①로 가면서 보정을 걸 자리가 사라졌다 → ②는 관측을 읽지 않는다 → 출력 4가지

그 위에 쌓인 것

1층 · 기원

위반 보정 누락

배분이 관측에 반응 못 함 → SLA 위반의 60~90%가 배분 탕

2층 · 증폭

개입 스텝 귀속 오류

폴백 상수의 실패가 정책 점수로 → r 0.50 → 0.20

3층 · 지표 무효

개입 판정 공식

선행 위험 신호가 없음 → AUC 0.49~0.68 → 논문 지표 무의미

개입 → 폴백 실패 → r↓ → 더 개입

근거 — 전부 실서버 실측

관측	값	뜻
SLA 위반 중 배분 탃 (나머지는 압력 >1.0)	19/21 · 12/20	normal · emergency 30스텝. 조달이 아니라 배분이 관측에 반응하지 않아서
개입 스텝의 적용 배분 = 폴백 상수	18/18 · 19/19	성적표의 60%가 정책이 하지 않은 일의 점수
rule_based 신뢰도 r	0.500 → 0.200	120스텝 동안 붕괴 — 정책이 나빠서가 아니라 개입했기 때문
combined 의 SLA 예측력	0.683 / 0.487	normal / emergency. 0.5 = 무작위
intrinsic 단독	0.487 / 0.463	분기 명확성을 쟁 쏴 위험이 아님
worst u/θ (관측에 있는 값)	0.857 / 0.608	가장 좋은 신호가 공식 밖에 있다
lstm_forecast 선택 → 개입	47 / 47	표본이 영원히 0. 정책 선택 실증 불가
조달 제약(1.0) 전 → 후	5회 → 1회	스텝당 \$0.109 → \$0.082. 두 드라이버 비교가 공정해짐

먼저 정해야 할 것 — 코드보다 먼저

D1 · 위반 보정

원본 :446~457 . 임계를 넘은 슬라이스에 더 주고 여유 있는 데서 뻔다. kim이 "Day 0에 3인이 정할 사항"으로 남겨둔 것.

권고 환경변수 스위치. 기존 기준은 컴(원본 동작). 끈 값은 상황인지 순도 논증용.

D2 · 개입 판정 공식

$\sqrt{(\text{intrinsic} \times \text{empirical})} < 0.45$. 원본에 없던 신규 설계. 두항 모두 선행 위험 신호가 아니다.

권고 귀속 오류(B-2)를 고친 뒤 다시 **젠 값**으로 정한다. $\tau=0.45$ 는 유지하고 민감도를 보고.

D3 · 조달 시점

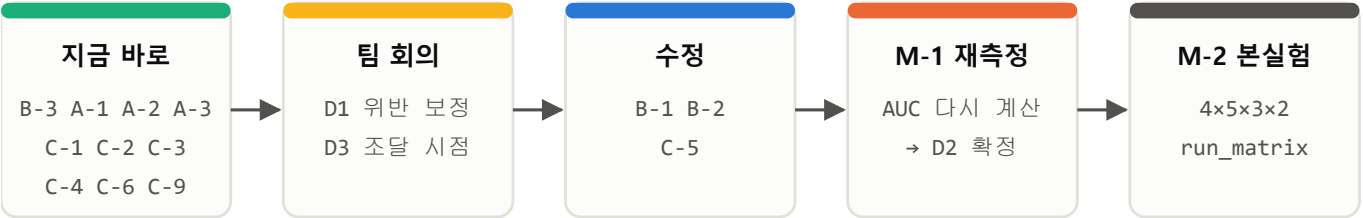
1.0을 넘은 뒤 사면 용량은 다음 스텝에 반영 → 그 스텝 위반 확정. 반응형이다.

권고 고정 루프는 1.0 유지(기존 기준), 오케스트레이터만 추세 선제 허용 → 그 차이가 기여. 단 D1 이후.

누가 무엇을

B · kim · ②③⑤ B-1 policy/rule.py 위반 보정 이식 ← D1 B-2 feedback/server.py:143 개입 스텝은 r 갱신 제외 B-3 feedback/reliability.py:60 lstm 봉인 해제 B-4 rule.py "안전 하한" 독스트링 정정	A · choi · ①④ A-1 audit/book.py:234 agent_policy 필드 A-2 audit/book.py confidence 타입 검사 A-3 observe/ Observation 예 features 블록 — classify_demand 가 매 스텝 죽는다	C · lee · 에이전트 C-1 intent 를 장부에 기록 C-2 eval/score.py 구조적·오류 스텝 분리 C-3 referee.py 개입 무시 규칙 C-4 arms/ 3종 · escalate 필드화 C-5 tools/run_matrix.py C-6 vendors.json 추적 해제 C-7 개입 공식 ← D2 · M-1 C-8 선제 조달 프롬프트 ← D3 · B-1 C-9 run_matrix orch-llm 칸 시드당 반복 — 같은 시드에서 실행마다 다르다
---	---	--

순서



D2(개입 판정 공식)는 M-1 결과를 보고 정한다 — 귀속 오류가 견힌 뒤의 값이어야 한다

지금 바로 줄은 반나절이고 서로 독립이다. 회의 전에 끝내두면 D1이 나는 순간 B-1 → M-1로 간다. 결정 전에 본실험 60회를 돌리면 전부 다시 돌리게 된다.

부록 · 작업 계획 원문

claude/team/workplan.md — AI 가 그대로 실행할 수 있게 쓴 것

작성 2026-09-24 · C(에이전트) · 대상 A·B·C 전원과 이 저장소에서 일하는 AI

0. 이 문서를 읽는 AI 에게

목적. 논문 주장 "개입은 줄었는데 SLA 위반은 늘지 않았다" 를 측정 가능하게 만든다. 지금은 측정이 불가능하다. 이 문서는 그 이유와, 가능해질 때까지의 작업을 담당·파일·줄·검증 명령까지 적는다.

전제. - 실행은 .venv310\Scripts\python.exe 로만 한다. PATH 의 3.14 에는 fastmcp · tensorflow 가 없다. - 담당 표기: **A**=choi(①④·측정) · **B**=kim(②③⑤) · **C**=lee(에이전트· eval · tools) - **자기 담당이 아닌 파일은 고치지 않는다.** 여기 적힌 변경은 해당 담당이 한다. - 근거 수치는 runs/ 의 실행 산출물에서 나왔다. 재현 명령은 각 항목에 있다. - 이 문서보다 자세한 원인 분석은 issue-B-policy-feedback.md (8건) 에 있다. 우선순위·담당 표는 todo-after-orchestrator.md (14건) 에 있다. 이 문서는 그 둘을 **실행 순서**로 다시 엮은 것이다. 셋이 어긋나면 이 문서가 최신이다.

하지 말 것. - 결정 D1·D2·D3 이 나기 전에 본실험(60회)을 돌리지 않는다. 전부 다시 돌리게 된다. - 결과가 좋아질 때까지 상수(τ ·평활·임계)를 돌리지 않는다. 민감도로 보고한다. - --intent 에 상황 라벨("지금 emergency 다")을 주 실험에 넣지 않는다. 오라클 비교군에만.

1. 진단 — 세 층

- 1층

분해 과정에서 피드백 루프가 끊겼다
원본 ml_orchestrator_demo.py:446~457 (위반 보정) 이
a(평활·클립) 과 b(목표표) 사이에 빠짐
→ a.propose() 가 observation 을 안 읽음 → 출력 4가지
→ SLA 위반의 60~90% 가 배분 탕 (나머지는 압력>1.0 · 조달로만 해소)

← 기원
- 2층

실패가 영동한 주체에게 귀속된다
개입 스텝은 폴백 상수 {0.4,0.4,0.2} 를 적용하는데
c server.py:143 이 그 결과를 rule_based 의 r 에 기록
→ 개입 → 폴백 실패 → r↓ → 더 개입 (자기강화)
→ r 0.500 → 0.200 (120스텝)

← 증폭기
- 3층

개입 판정 공식이 위험을 재지 않는다
combined = $\sqrt{\text{intrinsic} \times \text{empirical}}$
intrinsic(rule) = $0.5 + 0.3 \cdot \min_k |u_k - \theta_k| / \theta_k$ → 분기 명확성. 위험 아님
empirical = EMA(과거 sla_met) → 후행 지표
→ 실패를 예측 못 함 (AUC 0.487~0.683)
→ 관측에 있는 worst u/ θ 는 AUC 0.857 인데 공식 밖

← 논문 지표 무효

근거 (전부 실서버 · 재현 가능)

관측	값	실행
SLA 위반 중 배분 탕 / 구조적 (압력>1.0)	19·2 (normal) · 12·8 (emergency) · 4·1 (mixed 12스텝)	measure-*-s0 30스텝 · prompttest-mixed-s0 . python -m eval.breakdown <id>
개입 스텝의 적용 배분 = 폴백	18/18 · 19/19	measure-normal-s0 · measure-emergency-s0

관측	값	실행
rule_based r 붕괴	0.500 → 0.200	120스텝 mixed (뒹어써짐, 리포트에 기록)
combined 의 SLA 예측력 (AUC)	0.683 / 0.487	normal / emergency 30스텝
intrinsic 단독	0.487 / 0.463	같음
worst u/θ (관측값)	0.857 / 0.608	같음
lstm_forecast 선택 → 전 부 개입	47/47, n 은 0	120스텝 mixed
조달 제약(1.0) 전후	5→1회 · \$0.109→\$0.082/스텝	12스텝 orchestrator ×2

재현:

```
.venv310\Scripts\python.exe run.py --backend mcp --decider rule --scenario emergency --steps 30 --arm
measure --fresh --memory-mode cold
.venv310\Scripts\python.exe -m eval.score measure-emergency-s0
```

(AUC 는 decisions.json 의 confidence.combined 와 outcome.sla_met 로 계산한다. 스크립트는 \$6 참조.)

2. 먼저 정해야 할 것 — 코드 전

D1. 위반 보정을 되살릴 것인가

무엇	원본 :446~457 — 임계 넘은 슬라이스에 +min(0.1, 초과분×0.2), 가장 여유 있는 슬라이스에서 뺌
왜 빠졌 나	policy/rule.py 모듈 독스트링 — 평활이 ①로 가서 걸 자리가 사라짐. "Day 0 에 3인이 정할 사항"
켜면	배분이 관측에 반응. SLA 위반 63~92% 가 내려갈 것
안 켜면	상황 인지의 가치를 측정할 수 없다. 정답을 아는 baseline 과 55% 맞는 arm1 의 SLA 가 22/30 으로 같다 (C-4 실측). 사다리 첫 칸이 죽는다
끄면	상황 라벨 오판이 보정으로 가려지지 않아 상황인지 신호가 깨끗함
권고	환경변수 스위치 SLICE_RULE_CORRECTION=on/off . 기존 기준은 on (원본 동작). off 는 상황인지 순도 논증용 대 조
여는 작 업	B-1

D2. 개입 판정 공식을 어떻게 할 것인가

무엇	combined = √(intrinsic × empirical) < τ(0.45) — 설계서 flow/data-chain.md:107 . 원본에는 없던 신 규 설계
문제	두 항 모두 선행 위험 신호가 아님 (\$1 3층). AUC ~0.5

주의	2층(귀속 오류)이 empirical 을 오염시키고 있음. B-2 이후 재측정(M-1)한 값으로 판단한다
선택지	(a) 유지하고 민감도만 보고 (b) worst u/θ 같은 선행 신호를 게이트로 추가 (c) 공식 교체
권고	M-1 결과로 정한다. 재측정 AUC 가 0.7 이상이면 (a). 0.6 미만이면 (b). 어느 쪽이든 $\tau=0.45$ 는 유지하고 0.3~0.6 민감도를 같이 보고
여는 작업	C-7

D3. 조달 시점

무엇	PROCURE_PRESSURE = 1.0 (spec/observe.md:27). 압력이 1.0 을 넘은 뒤 사면 용량은 다음 스텝에 반영 → 그 스텝 위반 확정
선택지	(a) 양쪽 다 1.0 반응형 (b) 고정 루프 1.0 유지 · 오케스트레이터는 이력 추세로 선제 허용
권고	(b). 고정 루프는 기존 기준 그대로, 오케스트레이터에만 재량 → 그 차이가 기여. 단 B-1 이후에 켜다 (지금은 위반의 60~90% 가 조달과 무관해 효과가 안 보임)
여는 작업	C-8

D4. 개입하면 무엇이 적용되는가 (신규 · M-0 에서 드러남)

무엇	개입 스텝에 적용되는 배분. 지금은 INIT_ALLOCATION 상수 — 실험 속 사람은 늘 평시 배분을 한다
문제	proposed 가 SLA 를 arm2 보다 7.3%p 나쁘게 만든다(14/15쌍). 개입 = 별점이 되어 주장이 "덜 부를수록 좋다"로 줄어든다
선택지	(a) 상수 유지 (b) baseline 행동 (c1) 현재 관측 전문가 $a*(obs_t)$ (c2) 완전 오라클 $a*(obs_{\{t+1\}})$
권고	(c1). "필요한 스텝만 골라 올린다"는 서사와 맞고, 개입 정밀도가 곧 기여가 된다. (c2)는 사람이 다음 스텝 트래픽까지 아는 가정이라 심사에서 공격받는다. ④ 코드라 A 소유 · 팀 결정. 상세는 §4b

3. 작업 목록

형식: ID · 담당 · 파일:줄 → 변경 → 검증 → 의존

B (kim · ②③⑤)

B-1 · srm_mcp/policy/rule.py propose() — 의존: D1 → 원본 :446~457 을 옮긴다. observation["utilization"] 과 THRESHOLDS 로 임계 초과 슬라이스에 $increase = \min(0.1, (u-\theta) \times 0.2)$ 를 더하고 utilization 이 가장 낮은 슬라이스에서 뺀다. SLICE_RULE_CORRECTION 환경변수(기본 on)로 켜고 끈다. 평활·클립은 넣지 않는다(①의 몫). → 검증: emergency 30스텝에서 propose_allocation 반환 allocation 이 스텝마다 달라진다 (지금은 4가지 상수). SLA 위반이 21/30 아래로 내려간다. → 주의: 원본은 평활된 배분에 보정을 걸었다. ②는 현재 배분을 모르므로 목표표에 직접 건다. 이 차이를 독스트링에 적는다.

B-2 · srm_mcp/feedback/server.py:143~163 — 의존: A-1 (권장, 필수 아님) → `record.get("escalated")` 가 참이면 `reliability.update()` 와 `push_error()` 를 건너뛰다. `outcome` 은 그대로 돌려준다(④ 채점은 남아야 함). 폴백 성능을 따로 보려면 `table["fallback"]` 항목을 두고 거기 기록한다(선택). → 검증: `measure-emergency-s0` 재실행 후 `reliability.json` 의 `rule_based.n` 이 개입 횟수만큼 적다. 개입 19회면 `n=11`. → 근거: `issue-B-policy-feedback.md` 5번.

B-3 · srm_mcp/feedback/reliability.py:53~61 — 의존: 없음 → `errors` 가 비면 `None` 대신 `reliability` 사전값에 서 유도한 오차를 준다. `reliability.py` 의 `initial_entry()` 가 `r=0.5` 를 주므로, 정책별 사전값이 필요하다면 `POLICY_PRIOR = {"rule_based":0.9, "lstm_forecast":0.8, "dqn":0.4}` 를 `const.py` 에 두고 `recent_error = 1 - prior` 로 유도한다 (`lstm → 0.2 → ②  $\text{conf exp}(-3 \times 0.2) = 0.549 > \tau$`). → 검증: `normal` 30스텝 규칙 판단자에서 스텝 10 이후 `lstm_forecast` 의 `confidence` 가 0.2231 이 아니다. 실행 끝에 `lstm_forecast.n > 0`. → 근거: 같은 문서 1번. `reliability.py:57` 의 유예 가정에 구멍이 있다(`errors` 는 정책별).

B-4 · srm_mcp/policy/rule.py confidence() 독스트링 — 의존: 없음 → "하한 0.50 이라 항상 τ 위"를 지운다. `combined` 는 `empirical` 이 곱해지므로 보장이 아니다. → 근거: 같은 문서 2번.

A (choi · ①④)

A-1 · srm_mcp/audit/book.py:172~250 — 의존: 없음 → `record_escalation` 시그니처에 `chosen_policy`: `Optional[str] = None` 을 받고, 폴백 결정 레코드에 `"agent_policy": chosen_policy` 를 추가한다. `"chosen_policy": FALLBACK_POLICY` 는 그대로 둔다(⑤가 실행된 정책을 알아야 함). `situation` 을 보존한 :185 의 논리와 같다. → 검증: 오케스트레이터 12스텝 후 `decisions.json` 의 개입 레코드에 `agent_policy` 가 있고, 루프 요약의 정책 사용 과 장부의 `agent_policy` 집계가 일치한다. → 근거: 같은 문서 4번. 에이전트(C-1)가 값을 넘긴다.

A-2 · srm_mcp/audit/book.py bad_confidence() — 의존: 없음 → 네 키의 값이 `float` 로 변환되는지 확인한다. 아니면 `malformed_confidence` 로 거부. → 근거: `todo-after-orchestrator.md` 8번 (haiku 가 문자열을 넣은 실측).

A-3 · srm_mcp/observe/ Observation 에 features 블록 — 의존: 없음 · 신규 2026-09-25 → `classify_demand` 가 매 스텝 `feature_mismatch` 로 죽는다 — `Observation` 만으로는 11차원 (`traffic_load · time_of_day · *_alloc · *_util · client_count · bs_count`)을 못 만든다. ①이 `observation["features"]` 를 실어 주면 ②의 `features.py:59` 가 그대로 읽는다. 형태는 `handover.md` §(2) 의 JSON 그대로. → 왜 지금: 오케스트레이터 12스텝 (`orchcheck2-special_event-s0`)에서 LLM 이 실패한 `classify_demand` 를 11개 피처를 스스로 조립해서 다시 불렀고 (eMBB 99.9%), 그 결과로 상황을 `special_event` 로 잡았다. 피처 정의를 하나라도 틀리면 분류가 엉터리인데 지금은 확인할 길이 없다. 같은 시드의 직전 실행은 재시도를 안 해 `normal` 12/12 였다 — **DQN 입력을 LLM 이 만드느냐 ①이 만드느냐가 상황 판단을 가른다.** 판단의 입력은 서버가 낸 값이어야 한다. → 검증: 오케스트레이터 12스텝에서 `classify_demand` 첫 호출이 `available: true` , 그리고 `calls.jsonl` 의 `classify_demand` 인자에 LLM 이 만든 평탄 키(`embb_alloc ...`)가 없다.

C (lee · 에이전트 · eval · tools)

C-1 · run.py · agent/loop.py · agent/orchestrator/{host,gateway}.py — 의존: 없음 · 완료 2026-09-24 → 실행 조건(`scenario · seed · arm · driver · intent · ...`)을 ④ `record_decision / record_escalation` 의 `config` 로 넘긴다. 고정 루프는 `loop.py` 가, 오케스트레이터는 게이트웨이 미들웨어가 LLM 의 기록 호출에 끼워 넣는다(호스트 값이 LLM 값을 이긴다). → 발견: 그 전까지 모든 장부에서 `scenario · seed · arm` 이 `None` 이었다. ④ `default_config()` 는 `SLICE_SCENARIO` 등 환경변수만 읽는데 아무도 놓지 않았다. → 검증: `decisions.json["config"]` 에 `scenario · seed · arm · intent` 키가 값으로 있다. → △ `chosen_policy` 중계는 **A-1 이후에** 넣는다. FastMCP 는 모르는 인자를 `unexpected_keyword_argument` 로 거부한다(실측). 미리 보내면 모든 개입 스텝이 죽는다.

C-2 · eval/breakdown.py 신규 — 의존: 없음 · 완료 2026-09-24 → `eval/score.py` 는 A 소유라 고치지 않고 감싼다. 그 공개 함수를 불러 둘을 덧붙인다. (a) SLA 위반을 배분 탭 / 구조적(압력>1.0) / 판정 불가로 나눈다. 제외가 아니라 분리. (b) `runs/<id>/orchestrator/referee.jsonl` 의 `error` 등급 위반 스텝을 빼고 두 지표를 다시 계산하며, 뺀 수를 함께 보고한다. → △ 구조적 판정은 `obs_{t+1}` 의 압력으로 한다. SLA 는 `step()` 뒤 관측으로 채점되므로 결정 레코드의

obs_t 를 쓰면 한 스텝 어긋난다 (rationale/audit.md 의 violations 함정과 같다). → 검증: python -m eval.breakdown measure-emergency-s0 → 배분 탭 12 · 구조적 8 · 판정 불가 1. → 근거: 같은 문서 8번 · todo 6번.

C-3 · agent/orchestrator/referee.py — 의존: 없음 · 완료 2026-09-24 → 기록 직전의 마지막 compute_confidence 판정과 실제로 부른 기록 도구를 대조한다. ignored_escalation (판정 true 인데 decision) · escalation_without_trigger (false 인데 escalation) · no_confidence_check (판정 없이 기록). 호출 기록에 추출된 escalate 를 쓴다. → △ severity 는 warn. 처음 계획한 error 는 틀렸다 — error 는 "채점이 불가능하거나 왜곡된 스텝"이고 C-2 의 breakdown.py 가 채점에서 뻐다. 이걸 기록·적용·보고가 멀쩡한 판단 실패라 **채점에 남아야** 한다. error 로 두면 가장 세야 할 스텝이 지표에서 사라진다. → 검증: tools/check_orchestrator.py 에 세 규칙 시험 추가, 전부 통과. 기존 합성 "표준 순서" 셋이 compute_confidence 를 빼고 있어 함께 고쳤다. → 소급: 지난 LLM 실행 3개(22스텝)에서 위반 0. 판정 true 3회 → 개입 3회. 표본이 작다. → 근거: todo 7번.

C-4 · agent/schema.py · agent/arms/ 신규 — 의존: 없음 · 완료 2026-09-24 → Decision.escalation: Optional[bool] 필드를 두고 escalate 는 그걸 우선한다(없으면 공식). arms/ — baseline.py (truth.jsonl 을 읽는 유일한 파일) · supervised.py (arm1-arm2) · __init__.py (선택). run.py --arm {baseline|arm1|arm2|proposed} . 그 외 라벨은 proposed 동작. → 해석: 비교군 1:2 의 에스컬레이션을 decomposition.md 는 "항상 사람 감시", roles.md 는 "없음" 으로 적는다. 뜻은 같다(L2 — 에이전트가 개입을 요청할 일이 없다). 코드상 escalation=False . 모든 스텝이 자기 판단으로 채점되어 V2 표본 편향이 없다. → 사다리: 각 비교군이 앞의 것에 능력 하나를 더한다(corrections.md:82 "정확히 한 변수만"). 그래서 **arm1 은 조달도 규칙으로 묶었다** — 판단자가 LLM 이면 조달도 LLM 이 하는데, 풀어두면 baseline 대비 두 변수가 바뀐다. △ 팀 확인 필요 (설계서 비교군 표는 조달 도입 전에 쓰였다). → 정정: 처음 계획한 "escalate 항상 True" · "baseline 만 Guard 끄" 은 틀렸다. 개입은 False 이고, baseline 의 정답은 파일에서 오지 도구 반환값으로 오지 않으므로 Guard 는 켜둔다. → 막는 조합: 오케스트레이터 + arm1/arm2/baseline (오케스트레이터는 정의상 proposed) · baseline + mock (정답 파일은 실서버 ① 이 쓴다). 서버를 띄우기 전에 거절한다. → 검증: 목 emergency 20스텝 — arm1-arm2 개입 0, proposed 12. 실서버 mixed 30스텝 — baseline 상황 인지 1.00 (29/29), arm1 0.55. agent/ 에서 truth 를 여는 코드는 baseline.py 뿐.

→ 발견 — 지금은 상황 인지의 가치가 0 이다. 같은 시드 mixed 30스텝:

상황 인지 SLA 위반 배분 탭 구조적 조달 baseline (정답) 1.00 22/30 16 5 5 arm1 (에이전트) 0.55 22/30 16 5 5

상황이 다른 13스텝에서 요청 배분이 달랐는데 SLA 는 26/30 스텝이 같고 나머지는 상쇄됐다. 목표표가 트래픽과 안 맞아 정답을 알아도 배분이 틀리기 때문이다(1층). 평할 0.7 이 차이를 더 깎는다. 사다리의 첫 칸(baseline→arm1)이 신호를 내지 못하므로 **B-1 전에 비교군을 돌리는 것은 의미가 없다**. D1 의 가장 직접적인 근거다.

C-5 · tools/run_matrix.py 신규 — 의존: C-4 · 완료 2026-09-24 → 변형 8종 × 시나리오 5 × 시드 3 = 120칸. 변형은 비교군 × 판단자다 — baseline · arm1_{rule,llm} · arm2_{rule,llm} · proposed_{rule,llm,orch} . 라벨 첫 토큰이 비교군 동작을 정하도록 arms.kind_of 를 넓혔다(판단자만 바뀌도 run_id 가 안 겹침). → --go 없이는 계획과 추정만 출력한다. 기본이 실행이 아니다. --budget-usd (기본 \$20, 구독 사용량 환산) 누적에 닿으면 다음 칸을 시작하지 않는다. --resume 은 state.json 에서 성공한 칸을 건너뛴다. 실패 칸(LLM 없음)부터 돈다. → 산출: runs/_matrix/<이름>/{plan,state,summary}.json · summary.csv · logs/ . 칸별 행 = 개입 수·룰 · SLA(배분 탭·구조적 분리) · 조달 · 상황 인지 · 개입 정밀도 · 심판 오류. → 전체 추정: 120칸 · 29.7시간 · 사용량 환산 \$253 (rule 60 · llm 45 · orch 15칸). → 검증: 무료 3칸(baseline · arm1_rule · proposed_rule, emergency, 10스텝)을 --go 로 끝까지.

C-6 · git — 의존: 없음 → git rm --cached data/vendors.json . .gitignore 에 이미 있다. 부트스트랩이 원본에서 재생성. → 검증: git status 가 실행 후에도 깨끗하다.

C-7 · agent/schema.py · agent/orchestrator/gateway.py:294~301 — 의존: D2 · M-1 → D2 결과대로. (b) 라면 두 곳에 같은 게이트를 넣는다 — 한쪽만 바꾸면 드라이버 비교가 깨진다. 단위 검증 스크립트가 두 구현이 같은 입력에 같은 출력을 내는지 확인한다. → 검증: check_orchestrator.py 의 compute_confidence 공식 항목이 새 공식으로 통과.

C-8 · agent/orchestrator/prompt.md — 의존: D3 · B-1 → (b) 라면 조달 절에 "get_history 의 추세가 상승이고 다음 스텝에 1.0 을 넘을 것으로 보이면 선제 조달할 수 있다" 를 넣는다. 고정 루프는 건드리지 않는다. → 검증: 압력 상승 구간에서 조달 스텝이 1.0 도달 전에 나타난다.

비교군	SLA 위반율	상황 인지	개입률	개입 정밀도
baseline	0.794 ±0.125	1.000	0	—
arm1	0.650 ±0.111	0.418	0	—
arm2	0.650 ±0.111	0.418	0	—
proposed	0.723 ±0.130	0.488	0.639	0.758

시드 간 표준편차 0.024~0.038 — 아래 차이(0.07~0.14)는 잡음이 아니다.

사다리 세 칸이 전부 막혀 있고, 칸마다 막은 것이 다르다 (같은 시나리오·시드 15쌍):

칸	SLA 위반율 차이	쌍	막은 것
상황 인지 (arm1-baseline)	-0.144 (정답이 더 나쁨)	좋아짐 12 · 같음 1 · 나빠짐 2	목표표 오교정 → B-1
정책 선택 (arm2-arm1)	±0.000	15쌍 전부 같음	lstm 봉인 → B-3
선택적 개입 (proposed-arm2)	+0.073 (개입이 해침)	좋아짐 1 · 나빠짐 14	폴백 상수 · 귀속 → B-2 · D4

목표표 오교정 — 모든 시나리오에서 URLLC 과배분. 다음 스텝 관측으로 ⑤ 식(a^*)을 계산해 표와 대조:

	목표표 - 실제 필요 (embb / urllc / mmtc)	거리
normal	-0.08 / +0.14 / -0.06	0.137
emergency	-0.13 / +0.23 / -0.09	0.227 ← 정답 칸이 가장 틀림
special_event	-0.02 / +0.11 / -0.09	0.114
iot_surge	-0.09 / +0.08 / +0.01	0.089

URLLC 임계가 1.2 로 가장 높아 실제로는 적은 몫이면 되는데 표는 0.3~0.7 을 준다. 원본에도 같은 표·임계가 있다(:174 , :429) — 원본은 위반 보정(:446~457)이 매 스텝 메웠을 것이다. 다만 보정은 스텝당 최대 +0.1 이고 평활이 걸려 **emergency 의 0.23 을 다 못 메울 수 있다.** B-1 후 M-0 재측정으로 확인한다.

D4 (신규) · 개입하면 무엇이 적용되는가

지금은 개입 시 `fallback_allocation = INIT_ALLOCATION {0.4, 0.4, 0.2}` 상수가 적용된다(`audit/book.py`). 즉 실험 속 "사람"은 늘 **평시 배분을 하는 사람**이다. proposed 가 64% 스텝에서 그 사람을 불렀고, 그래서 SLA 가 arm2 보다 7.3%p 나빠졌다. 특히 `special_event`(개입 89%) — 실제 필요 embb 0.62 에 0.4 를 준다.

논문 주장 "개입은 줄었는데 SLA 는 안 늘었다" 는 **개입이 무엇을 사오는지에 달려 있다.**

사람 모형	개입의 효과	주장의 뜻
(a) 상수 폴백 (현재)	늘 손해	"덜 부를수록 좋다" — 개입이 별점일 뿐
(b) baseline 행동 (정답 + 목표표)	목표표가 틀려 역시 손해	(a) 와 비슷
(c1) 현재 관측 전문가 $a^*(obs_t)$	부른 스텝이 대체로 좋아짐	"필요한 스텝만 골라 부른다" — 정밀도가 곧 기여
(c2) 완전 오라클 $a^*(obs_t+1)$	부른 스텝은 확실히 충족	같음. 다만 사람이 미래를 아는 가정

오라클 = 정답을 완벽히 아는 가상의 존재. 현실 성능이 아니라 "사람이 완벽할 때의 상한" 을 재는 장치다. a^* 는 ⑤ 가 채점에 이미 쓰는 식(`feedback/scoring.py` `ideal_allocation` — `traffic ÷ (θ × capacity)` 를 합 1로) 이라 새로 만들 것이 없다.

(c1) 을 권고한다. 채점은 `obs_{t+1}` 로 하므로 (c2) 는 사람이 다음 스텝 트래픽까지 안다는 가정이 된다. (c1) 은 지금 관측만 쓰되 정확히 계산하는 사람이라 미래 지식 없이 "개입하면 좋아진다"가 성립하고, 심사에서 공격받을 여지가 적다. 결과는 "현실 성능"이 아니라 "사람 모형 (c1) 하의 상한"으로 보고한다.

④ 코드(`book.py` `record_escalation` 의 `fallback`)를 바꾸는 일이라 A 소유이고 팀 결정이 필요하다. 어느 쪽이든 **논문에 사람 모형을 명시**해야 한다.

5. 완료 판정

각 단계가 끝났다고 말하려면 아래가 참이어야 한다.

단계	판정
B-1	<code>emergency</code> 30스텝 SLA 위반 $< 21/30$, <code>propose_allocation</code> 출력이 스텝마다 다름
B-2	개입 19회 실행 후 <code>rule_based.n == 30 - 19</code>
B-3	30스텝 후 <code>lstm_forecast.n > 0</code>
A-1	개입 레코드에 <code>agent_policy</code> 존재, 루프 집계와 일치
A-3	오케스트레이터 12스텝에서 <code>classify_demand</code> 첫 호출이 <code>available: true</code>
C-9	<code>summary.json</code> orch 행에 <code>n_repeats ≥ 2 · sla_violation_sd</code>
C-2	채점 결과에 <code>structural</code> 과 <code>excluded_by_referee</code> 키
C-4	<code>arms/baseline.py</code> 만 <code>truth.jsonl</code> 을 연다 (<code>grep -l truth agent/</code> 가 그 파일 하나)
M-1	<code>combined</code> AUC 가 두 시나리오에서 기록됨. 값이 무엇이든
D2	M-1 값과 선택 근거가 이 문서 §2 에 추가됨
M-2	<code>runs/matrix-*/summary.json</code> 에 60행 이상, 각 행에 개입률·SLA·비용·perception-escalation_precision

6. 참조

- 원인 상세: `claude/team/issue-B-policy-feedback.md` (8건, 실측 · 선택지)
- 우선순위 표: `claude/team/todo-after-orchestrator.md` (14건)
- 비교군 정의: `claude/origin/decomposition.md` §6
- 정답 차단: `claude/flow/forbidden.md`
- 실행법 · 환경: `agent/README.md` · `web/serve.py` (실행 UI)
- AUC 계산: `runs/<id>/decisions.json` 의 `kind=="decision"` 레코드에서 (`confidence.combined`, `outcome.sla_met`) 를 모아 Mann-Whitney U / (`n_pos·n_neg`). `worst u/θ` 는 `max_k observation.utilization[k] / thresholds[k]` .